ESP32 / ARDUINO · COURSE 3

First I2C, First Display

One pair of wires carries many devices, each at its own address. Read a BME280 and show it on an OLED — both on the same two wires.

ESP32-003 · ~40 MIN · BEGINNER

1. The one idea

I2C — one pair of wires (SDA + SCL) carries many devices, each answering to its own address. Wire a BME280 sensor and an OLED display onto the *same* two lines, and the ESP32 talks to each by name.

2. Why it matters

Almost every sensor and display you'll touch speaks I2C. Learn the bus once — two shared lines, device addresses, and one set of pull-up resistors — and dozens of parts just plug in. It is the single most reused wiring skill on the bench.

3. Parts & wiring

- ESP32 dev board
- BME280 (temperature / humidity / pressure, I2C)
- SSD1306 0.96" OLED (I2C)
- Two 4.7kΩ resistors for the bus pull-ups (if your modules don't already have them)

Line	ESP32	Goes to
SDA	GPIO21	BME280 SDA **and** OLED SDA (shared)
SCL	GPIO22	BME280 SCL **and** OLED SCL (shared)
VCC	3V3	both modules
GND	GND	both modules

> *One pull-up pair for the **whole bus** — a single 4.7kΩ from SDA→3V3 and one from SCL→3V3, not one per device. Many breakout boards already include them; stacking several in parallel makes the pull-up too strong, rounds off the edges, and the bus stops working. Decouple each module's VCC with 0.1µF.*

4. Predict (do this before uploading)

- Two devices share the same two wires — how does the ESP32 address just one?
- Run an I2C scan. How many addresses show up? Guess them (BME280 is 0x76 or 0x77; the SSD1306 OLED is usually 0x3C).
- If you swap SDA and SCL, what does the scan report?

5. Build it — scan first, then read and show

Always scan the bus before trusting it:

```
#include <Wire.h>

void setup() {
  Serial.begin(115200);
  Wire.begin(21, 22);                  // SDA=21, SCL=22
  Serial.println("Scanning I2C ... ");
  for (uint8_t a = 1; a < 127; a++) {
    Wire.beginTransmission(a);
    if (Wire.endTransmission() == 0)   // 0 = a device ACKed at this address
      Serial.printf("found 0x%02X\n", a);
  }
}
void loop() {}
```

Once the scan shows both devices, read the sensor and draw it (Adafruit_BME280 + Adafruit_SSD1306 from the Library Manager):

```
#include <Adafruit_BME280.h>
#include <Adafruit_SSD1306.h>

Adafruit_BME280 bme;
Adafruit_SSD1306 oled(128, 64, &Wire);

void setup() {
  Wire.begin(21, 22);
  bme.begin(0x76);                      // address from your scan
  oled.begin(SSD1306_SWITCHCAPVCC, 0x3C);
}

void loop() {
  oled.clearDisplay();
  oled.setCursor(0, 0);
  oled.setTextSize(1);
  oled.setTextColor(SSD1306_WHITE);
  oled.printf("Temp  %.1f C\n", bme.readTemperature());
  oled.printf("Hum   %.0f %%\n", bme.readHumidity());
  oled.printf("Pres  %.0f hPa\n", bme.readPressure() / 100.0);
  oled.display();
  delay(500);
}
```

The new idea is the **address**: every device on the bus listens, but only the one you named responds. Two wires, many parts.

6. Test against your prediction

The scan should print `0x3C` (OLED) and `0x76` (BME280), and the OLED should show live temperature, humidity, and pressure. Breathe on the sensor — humidity climbs within a second. Did the addresses match your §4 guesses?

7. What goes wrong (pre-loaded debugging)

- **Scan finds nothing** → missing pull-ups, or SDA/SCL swapped. The bus needs pull-ups to idle high.
- **Scan finds only one device** → an address collision (two parts at the same address) or one module unpowered.

- **OLED stays black** → wrong address (0x3C vs 0x3D), or `begin()` not called with `SSD1306_SWITCHCAPVCC`.
- **Readings are nonsense** → the module is a 5V board on a 3.3V bus, or the bus is long and unshielded. Keep I2C runs short.
- **Bus is flaky with several boards** → stacked pull-ups (module + your own). Use one pair total; remove the extras.

8. Checkpoint

You drove a shared I2C bus — a sensor and a display on two wires, each addressed and answering on its own. That is the backbone skill for almost every peripheral in the rest of this track: displays, sensors, real-time clocks, expanders — they nearly all ride this same bus.

9. Push further

- Add the 2004 LCD to the *same* two wires (it has its own address) — three devices, two lines.
- Bump the bus to 400 kHz (`Wire.setClock(400000)`) and watch the edges on the Analog Discovery 3.
- Read the BME280's pressure into an altitude estimate — same data, a new meaning.
